

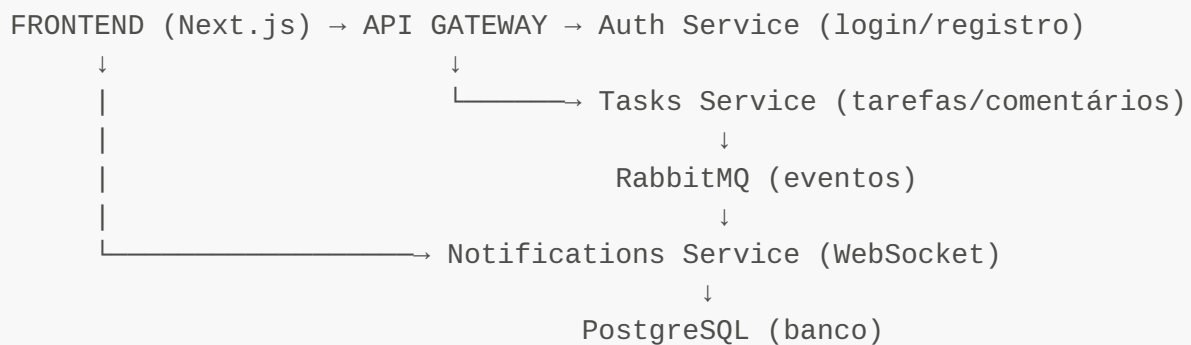
Task Manager - Guia Rápido

O que é este projeto?

Um sistema completo de **gerenciamento de tarefas** com:

-  Autenticação JWT (login/registro)
-  Criar, editar e deletar tarefas
-  Comentários em tarefas
-  Notificações em tempo real
-  Filtros por status e prioridade
-  Interface dark mode moderna

Arquitetura (4 Microserviços)



1 API Gateway (Porta 3001)

O que faz: Recebe todas as requisições do frontend e encaminha para o serviço correto.

Por que existe: Centraliza a entrada, valida dados e documenta APIs automaticamente.

Endpoints principais:

- `/api/auth/*` → Encaminha para Auth Service
- `/api/tasks/*` → Encaminha para Tasks Service

2 Auth Service (Porta 3002)

O que faz: Cuida de toda autenticação de usuários.

Funcionalidades:

- Registrar novo usuário
- Login (retorna 2 tokens JWT)
- Renovar token expirado

- Validar identidade do usuário

Tokens:

- **Access Token:** Válido por 15 minutos (usado em todas requisições)
 - **Refresh Token:** Válido por 7 dias (renova o access token)
-

3 Tasks Service (Porta 3003)

O que faz: Gerencia todas as operações de tarefas.

Funcionalidades:

- Criar tarefa (título, descrição, prioridade, data de vencimento)
- Listar tarefas (com filtros de status/prioridade/busca)
- Editar tarefa (só o autor pode)
- Deletar tarefa (só o autor pode)
- Adicionar comentários
- Atribuir usuários a tarefas

Status de Tarefa:

- **TODO** = A fazer
- **IN_PROGRESS** = Em progresso
- **DONE** = Concluído

Prioridades:

- **LOW** = Baixa
- **MEDIUM** = Média
- **HIGH** = Alta
- **URGENT** = Urgente

Extra: Quando algo acontece (tarefa criada, editada, etc), publica um evento no RabbitMQ.

4 Notifications Service (Porta 3004)

O que faz: Envia notificações em tempo real para os usuários.

Como funciona:

1. Fica escutando eventos do RabbitMQ
2. Quando recebe um evento (ex: "tarefa criada"), cria uma notificação
3. Envia via WebSocket para o usuário conectado no frontend
4. Salva no banco para histórico

Tipos de notificação:

- Nova tarefa criada
- Tarefa atualizada

- Tarefa deletada
- Você foi atribuído a uma tarefa
- Status da tarefa mudou
- Novo comentário adicionado

API REST: Também tem endpoints para listar/marcar como lida/deletar notificações antigas.

Como Rodar (PASSO A PASSO)

Pré-requisitos

Você precisa ter instalado:

- **Docker e Docker Compose**
 - **pnpm** (ou instale: `npm install -g pnpm`)
-

MÉTODO 1: Com Docker (Mais Fácil)

1. Suba todos os containers:

```
cd /home/devpads/Documents/desafio-globoom
sudo docker-compose up -d
```

2. **Aguarde ~30 segundos** para tudo inicializar.

3. **Pronto!** Acesse:

- **Frontend:** `http://localhost:3000` ← **Use este!**
- API Gateway Docs: `http://localhost:3001/api/docs`
- RabbitMQ Admin: `http://localhost:15672 (admin/admin)`

PROF

Para parar tudo:

```
sudo docker-compose down
```

Para ver logs:

```
sudo docker-compose logs -f
```

MÉTODO 2: Sem Docker (Desenvolvimento)

1. Suba apenas banco e RabbitMQ:

```
cd /home/devpads/Documents/desafio-globo
sudo docker-compose up -d db rabbitmq
```

2. Instale as dependências:

```
pnpm install
```

3. Crie os arquivos .env:

Backend (criar em cada serviço):

```
# apps/api-gateway/.env
PORT=3001
DATABASE_HOST=localhost
DATABASE_PORT=5432
DATABASE_USER=postgres
DATABASE_PASSWORD=password
DATABASE_NAME=challenge_db
RABBITMQ_URL=amqp://admin:admin@localhost:5672
JWT_SECRET=your-super-secret-jwt-key-change-in-production
JWT_EXPIRES_IN=15m
JWT_REFRESH_SECRET=your-super-secret-refresh-key-change-in-production
JWT_REFRESH_EXPIRES_IN=7d
CORS_ORIGIN=http://localhost:3000
```

Copie o mesmo .env para:

- apps/auth-service/.env (mude PORT=3002)
- apps/tasks-service/.env (mude PORT=3003)
- apps/notifications-service/.env (mude PORT=3004)

Frontend:

```
# apps/web/.env
NEXT_PUBLIC_API_URL=http://localhost:3001/api
NEXT_PUBLIC_WS_URL=http://localhost:3004
```

4. Abra 5 terminais e rode:

Terminal 1:

```
pnpm --filter api-gateway dev
```

Terminal 2:

```
pnpm --filter auth-service dev
```

Terminal 3:

```
pnpm --filter tasks-service dev
```

Terminal 4:

```
pnpm --filter notifications-service dev
```

Terminal 5:

```
pnpm --filter web dev
```

5. Acesse: <http://localhost:3000>



Testando o Sistema

1. Crie uma conta

- Acesse <http://localhost:3000>
- Clique em "Criar conta"
- Preencha email, username e senha
- Clique em "Registrar"

2. Faça login

- Entre com suas credenciais
- Você será redirecionado para o dashboard

3. Crie uma tarefa

- Clique no botão "Nova Tarefa" (canto superior direito)
- Preencha:
 - Título (obrigatório)
 - Descrição (opcional)
 - Status (TODO/IN_PROGRESS/DONE)
 - Prioridade (LOW/MEDIUM/HIGH/URGENT)
 - Data de vencimento (opcional)

- Clique em "Criar Tarefa"

4. Veja a notificação

- Olhe o sino 🔔 no header
- Deve aparecer um badge vermelho com "1"
- Clique no sino para ver a notificação em tempo real

5. Teste os filtros

- Use a busca para procurar tarefas
- Filtre por status (TODO, IN_PROGRESS, DONE)
- Filtre por prioridade

6. Edite uma tarefa

- Clique em "Editar" em qualquer tarefa
- Mude o status para "IN_PROGRESS"
- Salve
- Veja outra notificação aparecer

7. Adicione um comentário

- Clique em uma tarefa para ver detalhes
- (Implementação em progresso)



Stack Tecnológica

Backend:

- **NestJS** (framework Node.js)
- **TypeScript** (linguagem)
- **TypeORM** (ORM para banco de dados)
- **PostgreSQL** (banco de dados)
- **RabbitMQ** (fila de mensagens)
- **JWT** (autenticação)
- **Docker** (containers)

Frontend:

- **Next.js 16** (framework React)
 - **React 19** (biblioteca UI)
 - **TypeScript** (tipagem)
 - **Tailwind CSS** (estilos)
 - **Zustand** (gerenciamento de estado)
 - **Socket.IO** (WebSocket para notificações)
 - **Axios** (requisições HTTP)
-



Portas dos Serviços

Serviço	Porta	URL
Frontend	3000	http://localhost:3000
API Gateway	3001	http://localhost:3001
Auth Service	3002	Interno (não acesse diretamente)
Tasks Service	3003	http://localhost:3003
Notifications	3004	http://localhost:3004
PostgreSQL	5432	localhost:5432
RabbitMQ	5672	localhost:5672
RabbitMQ Admin	15672	http://localhost:15672



Como o Sistema Funciona (Fluxo Completo)

Exemplo: Criando uma tarefa

1. Você clica em "Nova Tarefa" no frontend
↓
2. Frontend envia POST /api/tasks para API Gateway
↓
3. API Gateway valida o JWT token
↓
4. API Gateway encaminha para Tasks Service
↓
5. Tasks Service salva no PostgreSQL
↓
6. Tasks Service publica evento "task.created" no RabbitMQ
↓
7. Notifications Service escuta RabbitMQ e recebe o evento
↓
8. Notifications Service cria notificação no banco
↓
9. Notifications Service envia via WebSocket para seu navegador
↓
10. Frontend recebe e mostra toast + atualiza badge do sino 🔔

PROF

? Troubleshooting

Problema: Erro de permissão no Docker

Solução: Use **sudo** antes dos comandos:

```
sudo docker-compose up -d  
sudo docker-compose down
```

Problema: Porta já em uso (3000, 3001, etc)

Solução: Veja qual processo está usando:

```
lsof -i :3000  
kill -9 <PID>
```

Problema: Frontend não conecta ao backend

Verificar: Arquivo `apps/web/.env` existe e tem:

```
NEXT_PUBLIC_API_URL=http://localhost:3001/api  
NEXT_PUBLIC_WS_URL=http://localhost:3004
```

Problema: Notificações não aparecem

Verificar RabbitMQ:

```
sudo docker-compose logs rabbitmq  
sudo docker-compose logs notifications-service
```

Problema: "pnpm: command not found"

Instalar:

```
npm install -g pnpm
```

Limpar tudo e recomeçar:

```
sudo docker-compose down -v  
sudo docker-compose up -d --build
```

Recursos Extras

Documentações Swagger (APIs interativas):

- API Gateway: <http://localhost:3001/api/docs>
- Tasks Service: <http://localhost:3003/api/docs>
- Notifications: <http://localhost:3004/api/docs>

RabbitMQ Admin (ver filas de mensagens):

- URL: <http://localhost:15672>
- Login: [admin](#) / [admin](#)

Conectar ao PostgreSQL (se precisar):

```
docker exec -it db psql -U postgres -d challenge_db
```



Paleta de Cores

- **Primária:** [#0A1828](#) (azul escuro)
- **Secundária:** [#178582](#) (verde água)
- **Destaque:** [#BFA181](#) (dourado suave)



Comandos Úteis

```
# Ver status dos containers
sudo docker-compose ps

# Ver logs de todos os serviços
sudo docker-compose logs -f

# Ver log de um serviço específico
sudo docker-compose logs -f api-gateway

# Parar tudo
sudo docker-compose down

# Parar e limpar banco de dados
sudo docker-compose down -v

# Reconstruir containers após mudar código
sudo docker-compose up -d --build

# Entrar dentro de um container
sudo docker exec -it api-gateway sh
```



Pronto! Sistema completo funcionando!

Se tiver dúvidas, os logs do Docker vão te ajudar: `sudo docker-compose logs -f`



Documentação técnica completa: Veja [DOCUMENTATION.md](#) para detalhes avançados.